

RAG 是把“你自己的资料库”接到大模型前面，让大模型先检索资料，再基于检索到的内容回答问题的一套工程流程。

一、这套内容适合什么场景？

这套 RAG pipeline 适合所有需要让大模型基于“私有数据、业务数据、文档资料、数据库内容”进行回答的场景。

典型场景包括：

1. 企业知识库问答

例如把公司制度、研究报告、产品文档、会议纪要、FAQ 放进去，让员工用自然语言查询。

2. 金融/投研资料助手

例如把券商研究报告、财报、行业数据、内部制度、KPI 规则等资料接入，让模型回答“某家公司收入结构如何”“某项考核规则依据是什么”。

3. 文档型 AI 助手

例如针对 PDF、Word、PPT、网页、数据库记录进行问答、总结、提取要点。

4. Agent 或 Chatbot 的知识增强模块

即使最终产品是聊天机器人或 Agent，也通常需要 RAG 作为“知识输入层”，让模型能使用外部资料，而不是只依赖自身训练知识。LlamaIndex 文档也明确提到，Query Engine、Chat Engine 和 Agent 经常会使用 RAG 来完成任务。

5. 需要降低幻觉、提高可追溯性的问答系统

RAG 的核心价值不是让模型“记住更多”，而是让模型在回答前先找到相关上下文，从而减少凭空编造。

二、RAG 的核心思想

大模型虽然在训练时学习了大量公开知识，但它 **没有天然学过你的私有数据**。RAG，也就是 Retrieval-Augmented Generation，解决的就是这个问题：先把你的数据加载、处理、索引起来；当用户提问时，系统先从索引中检索相关内容，再把这些内容连同问题一起交给大模型生成回答。

可以用一个通俗比喻理解：

普通大模型像一个知识很广的人，但他没看过你的公司资料。

RAG 就像先给他配一个“资料检索员”。

用户提问后，检索员先从资料库里找出最相关的几段材料，再交给大模型组织语言回答。

所以 RAG 的流程不是：

用户问题 → 大模型直接回答

而是：

用户问题 → 检索相关资料 → 把资料和问题一起交给大模型 → 大模型基于资料回答

LlamaIndex 文档中也把这个流程概括为：

数据先被加载并准备成可查询的 Index；

用户查询作用在 Index 上，Index 筛选出最相关的上下文；

这些上下文、用户问题和 Prompt 一起发送给 LLM，最后由 LLM 生成回答。

三、RAG Pipeline 的五个核心阶段

LlamaIndex 将 RAG 分成五个关键阶段：

Loading → Indexing → Storing → Querying → Evaluation

也就是：

加载数据 → 建立索引 → 存储索引 → 查询 → 评估

文档中明确列出 RAG 的五阶段：Loading、Indexing、Storing、Querying、Evaluation。

下面逐个说明。

1. Loading：加载数据

1.1 Loading 是什么？

Loading 是把原始数据从各种来源接入 RAG 系统。

数据来源可以是：

- 本地文本文件
- PDF
- Word
- PPT
- 网页
- 数据库

- API
- 图片、音频、视频等多模态文件

LlamaIndex 文档中说，Loading 指的是把数据从它所在的位置，例如文本文件、PDF、网站、数据库或 API，加载进工作流。

通俗理解：

Loading 就是“把资料搬进系统”。

例如你有一批 PDF 研报，第一步不是让大模型直接读，而是先用 Reader 或 Connector 把它们转换成 LlamaIndex 能处理的对象。

1.2 Document 和 Node

在 LlamaIndex 里，加载后的数据通常会变成两类对象：

Document：表示一个完整数据源，比如一篇 PDF、一份 API 返回、一条数据库查询结果。

Node：表示 Document 被切分后的小片段，是 LlamaIndex 中更细粒度的基本处理单元。文档中说，Document 是任何数据源的容器，而 Node 是 LlamaIndex 中的原子数据单元，代表源 Document 的一个 chunk，并带有与原文档和其他节点相关的元数据。

通俗理解：

Document 像一本书。

Node 像书里的一小段、一页或一个知识卡片。

RAG 检索时通常不是把整本书塞给大模型，而是找出最相关的几个 Node。

1.3 Reader / Connector

LlamaIndex 使用数据连接器，也叫 Reader，把不同来源和格式的数据加载成 Document 或 Node。

例如：

- 用 SimpleDirectoryReader 读取本地文件夹
- 用数据库 Reader 查询 SQL 数据库
- 用 LlamaHub 中的 Connector 连接外部系统

文档中提到，SimpleDirectoryReader 是最简单的 Reader，可以把指定目录下的文件变成 Document，并支持 Markdown、PDF、Word、PPT、图片、音频、视频等格式。

2. Transformation：数据转换与预处理

虽然 LlamaIndex 把它归在 ingestion/loading 过程中，但从工程角度看，它是非常关键的一步。

2.1 为什么要 Transform？

原始文档通常不能直接用于 RAG。原因包括：

- 文档太长，无法一次塞进模型上下文
- PDF/网页中可能有噪声，如页眉、页脚、目录、广告
- 文档缺少元数据，后续难以过滤
- 需要把文本切成适合检索的块
- 需要为每个文本块生成 embedding

LlamaIndex 文档中说，数据加载后，需要在放入存储系统前进行处理和转换，包括 chunking、提取 metadata、为每个 chunk 生成 embedding，这样数据才能被检索并被 LLM 更好使用。

通俗理解：

Loading 是把资料搬进仓库；

Transformation 是把资料拆箱、分类、贴标签、切成方便查找的小卡片。

2.2 Chunking：把文档切成 Node

RAG 系统中很重要的一步是把文档切成 chunks，也就是 Node。

文档中说，切分文档的核心思想是把数据处理成“bite-sized pieces”，也就是可以被检索、可以喂给 LLM 的小块。

通俗解释：

如果你把一本 100 页的 PDF 整体交给检索系统，它很难准确知道哪一页和问题相关。

但如果你先把它切成 500 字左右的小块，每个小块都有自己的语义向量，系统就能更精准地找到相关内容。

这一步会直接影响 RAG 效果。

chunk 太大，检索结果可能不够聚焦；chunk 太小，语义上下文可能断裂。

2.3 Metadata：给资料加标签

Metadata 是附着在 Document 或 Node 上的结构化信息，例如：

- 文件名
- 来源
- 作者
- 日期
- 分类
- 页码
- 业务条线
- 文档类型
- 权限等级

LlamaIndex 文档提到，可以手动或自动给 documents 和 nodes 添加 metadata。

通俗理解：

Metadata 就像图书馆书籍的标签：书名、作者、分类号、出版年份。
有了标签，后续不仅可以做语义检索，还可以做条件过滤。

例如在金融场景中，你可以给 Node 加上：

```
source = "2024年中期策略报告"  
asset_class = "equity"  
industry = "证券"  
date = "2024-06"
```

用户问“只看 2024 年的证券行业观点”时，就可以先用 metadata 过滤，再做语义检索。

3. Indexing：建立索引

3.1 Index 是什么？

在 LlamaIndex 中，Index 是一种由 Document 或 Node 组成的数据结构，用来支持 LLM 查询。文档中说，Index 是由 Document 对象组成的数据结构，目的是让 LLM 能够查询这些对象，并且 Index 的设计应当和查询策略互补。

通俗理解：

Index 就是为你的资料建立“可检索结构”。
没有 Index，大模型不知道该去哪里找资料；有了 Index，系统就能根据问题快速定位相关内容。

3.2 Vector Store Index

最常见的 Index 是 **VectorStoreIndex**。

它的流程是：

1. 接收 Documents
2. 把 Documents 切分成 Nodes
3. 为每个 Node 生成 embedding
4. 把 embedding 存入向量索引
5. 查询时根据语义相似度找出最相关的 Node

LlamaIndex 文档说，VectorStoreIndex 是最常见的 Index 类型，它会把 Documents 切成 Nodes，然后为每个 node 的文本创建 vector embeddings，供 LLM 查询。

3.3 Embedding 是什么？

Embedding 是 RAG 的核心概念之一。

文档中解释，embedding 是文本语义的数值表示；两个含义相近的文本，即使用词不同，也会有数学上相似的 embedding。

通俗理解：

Embedding 就是把一段文字转换成一串数字坐标。

意思相近的句子，在这个“语义空间”里的位置也会更近。

例如：

句子 A：这家公司盈利能力提升。

句子 B：该企业利润率有所改善。

虽然字面不同，但语义相近，所以 embedding 距离会比较近。

这就是为什么 RAG 能做“语义搜索”，而不只是关键词匹配。

3.4 Top-K Retrieval

当用户提问时，系统也会把用户问题转换成 embedding，然后和资料库中每个 Node 的 embedding 比较相似度，找出最相似的前 K 个结果。

文档中说，VectorStoreIndex 会将查询也转成 vector embedding，然后通过数学运算对所有 embeddings 按语义相似度排序；排序后返回最相似的 embedding 对应的文本块，这个返回数量就是 k，因此叫 top-k semantic retrieval。

通俗理解：

Top-K 就是“找出最相关的前 K 段资料”。

比如 top_k=5，就是从知识库里找出最相关的 5 个文本片段。

3.5 Summary Index

除了 VectorStoreIndex，LlamaIndex 还提到 **Summary Index**。

Summary Index 更适合做总结类问题。它会简单地存储所有 Documents，并把它们都返回给 query engine。

适合场景：

- 总结一篇文章
- 总结一组文档
- 对较短资料做整体归纳

不太适合：

- 大规模知识库精准问答
 - 多文档复杂检索
 - 对成本和上下文长度敏感的场景
-

4. Storing：存储索引

4.1 为什么要存储？

如果每次启动系统都重新加载文档、切分文本、生成 embedding，会非常慢，也可能很贵。

LlamaIndex 文档明确说，一旦数据被加载并建立索引，通常希望把它存储起来，以避免重复 re-index；默认情况下，索引数据只存在内存里。

通俗理解：

建索引就像给一整栋图书馆做编目。

这件事做一次就够了，没必要每次有人来问问题都重新编目。

4.2 Persist 到本地磁盘

最简单的存储方式是使用 `.persist()` 方法，把索引写入磁盘。之后可以通过 `load_index_from_storage` 重新加载，避免重新加载和重新索引。

需要注意的是，如果当初建立 index 时用了自定义 transformation、embedding model 等配置，重新加载时也要使用相同配置，或者设置为全局配置。

4.3 Vector Store

对于 VectorStoreIndex，embedding 的生成通常既耗时又有成本，所以更需要存储。

LlamaIndex 支持很多 vector store，文档示例使用了 Chroma。使用流程包括：

1. 初始化 Chroma client
2. 创建 collection
3. 将 Chroma 指定为 StorageContext 的 vector_store
4. 用这个 StorageContext 初始化 VectorStoreIndex

文档中说明，VectorStoreIndex 中创建 embeddings 的 API 调用可能耗费时间和金钱，因此应当存储它们，避免反复 re-index；示例中使用 Chroma 作为开源 vector store。

通俗理解：

Vector Store 就是专门存 embedding 的数据库。

普通数据库擅长查“字段等于什么”；向量数据库擅长查“语义上最像什么”。

4.4 新增文档

如果已经创建了 index，后续可以通过 `insert` 方法向 index 中添加新的 documents。

这在实际系统里很重要。因为知识库通常不是一次性建完的，而是不断更新：

- 新增制度文件
- 新增研究报告
- 新增产品说明

- 新增客户问答记录

5. Querying：查询

5.1 Querying 是什么？

Querying 是 RAG 中最接近用户的一层，也是 LLM 应用最关键的部分。

LlamaIndex 文档说，当你已经加载数据、建立索引并存储索引后，就可以进入 LLM 应用中最重要的一部分：querying。最简单的 querying 就是向 LLM 发起 prompt 调用；复杂的 querying 可以包括多次 chained prompt + LLM 调用，甚至跨多个组件的推理循环。

通俗理解：

Querying 就是用户真正开始问问题时，系统如何检索、加工、组织并生成答案。

5.2 QueryEngine

LlamaIndex 中查询的基础是 **QueryEngine**。最简单的方式是让 index 直接创建一个 query engine：

```
query_engine = index.as_query_engine()
response = query_engine.query("你的问题")
```

文档中也给出了类似示例：通过 `index.as_query_engine()` 获取 QueryEngine，然后调用 `query_engine.query(...)`。

5.3 Querying 的三个阶段

LlamaIndex 把查询过程拆成三个阶段：

1. **Retrieval：检索**
2. **Postprocessing：后处理**
3. **Response Synthesis：答案合成**

文档中明确说，querying 包含 retrieval、postprocessing 和 response synthesis 三个阶段。

5.3.1 Retrieval：检索

Retrieval 是根据用户问题，从 Index 中找出最相关的文档或节点。

最常见的是前面提到的 top-k semantic retrieval，也就是找出语义最相关的前 K 个文本块。文档中说，retrieval 是从 Index 中为 query 找到并返回最相关 documents 的过程，最常见的是 top-k semantic retrieval。

通俗理解：

Retrieval 是“查资料”。
用户问什么，系统先去知识库找最相关的几段。

5.3.2 Postprocessing：后处理

Postprocessing 是对检索出来的 Node 再做一层过滤、重排或增强。

例如：

- 只保留相似度超过 0.7 的结果
- 只保留带有某些关键词的 Node
- 排除某些关键词
- 根据相邻 Node 补充上下文
- 用 reranker 重新排序

文档中说，postprocessing 会对检索到的 Nodes 做 rerank、transform 或 filter，例如要求它们带有特定 metadata。

文档还列举了几类 Node Postprocessor：

- `KeywordNodePostprocessor`：根据 required keywords 和 exclude keywords 过滤
- `SimilarityPostprocessor`：根据相似度阈值过滤
- `PrevNextNodePostprocessor`：根据 Node 之间的关系补充前后文

通俗理解：

Retrieval 是先粗筛一批可能相关的资料；
Postprocessing 是再做精筛，把不够相关、质量不高或不符合条件的内容剔除。

5.3.3 Response Synthesis：答案合成

Response Synthesis 是把用户问题、检索到的相关资料、Prompt 一起交给 LLM，让 LLM 生成最终答案。

文档中说，response synthesis 是把 query、最相关的数据和 prompt 组合起来发送给 LLM，然后返回响应。

通俗理解：

检索系统负责“找材料”；
大模型负责“读材料、组织语言、回答问题”。

这一步决定了回答的风格、结构和细节程度。

5.4 Response Mode：不同答案生成模式

LlamaIndex 提供多种 response synthesis 模式。文档中列出包括：

- default
- compact
- tree_summarize
- no_text
- accumulate

其中：

default：逐个 Node 生成并 refine 答案，适合更细致的回答，但可能会有多次 LLM 调用。

compact：尽可能把多个 Node 塞进一次 prompt，减少调用次数。

tree_summarize ：把多个 Node 递归总结成树状结构，适合总结任务。

no_text：只执行 retriever，不真正调用 LLM，可以用来调试检索结果。

accumulate：对每个 Node 分别执行同一个 query，再把结果累积起来，适合需要逐块处理的任务。

通俗理解：

response mode 决定“大模型怎么读检索结果”。
是逐段细读？尽量压缩后一次读？还是做树状总结？不同模式适合不同任务。

6. Evaluation：评估

这几份材料里对 Evaluation 展开不多，但 Introduction 中明确把 Evaluation 列为 RAG 的第五个关键阶段，并说明它用于检查流程相对其他策略或变更后的效果，提供对准确性、忠实性和速度的客观衡量。

在实际 RAG 项目中，Evaluation 非常关键。因为 RAG 系统不是“能跑起来”就够了，还要判断：

- 检索到的内容是否相关？
- 回答是否忠实于原文？
- 有没有编造？
- 响应速度是否可接受？
- top_k 设置是否合适？
- chunk_size 是否合适？
- embedding model 是否合适？
- rerank 是否提升效果？

通俗理解：

Evaluation 就是给 RAG 做质检。

不评估，就不知道系统是真的变好，还是只是看起来能回答。

四、几个关键概念的通俗解释

1. RAG

RAG = Retrieval-Augmented Generation，检索增强生成。

一句话：

先检索资料，再让大模型基于资料回答。

它解决的问题是：大模型不知道你的私有资料，也可能胡编，所以要先把相关资料找出来作为上下文。

2. Document

Document 是一个完整数据对象。

比如：

- 一篇 PDF

- 一个网页
- 一条数据库查询结果
- 一个 Word 文档

它是 RAG 系统接入数据时的基本容器。

3. Node

Node 是 Document 被切分后的小块。

比如一篇 20 页 PDF 会被切成几十个 Node。

实际检索时，系统通常找的是 Node，而不是整篇 Document。

4. Chunking

Chunking 是把长文档切成小块。

它的目标是让每个小块：

- 足够短，可以放进模型上下文
 - 足够完整，保留必要语义
 - 足够精确，方便检索
-

5. Embedding

Embedding 是把文本变成数字向量。

比如一段话会被表示成几百维或几千维的数字。

语义越相近，向量距离越近。

6. Vector Store

Vector Store 是专门存储和检索 embedding 的数据库。

它不只是查关键词，而是查“语义相似度”。

7. Index

Index 是让资料变得可查询的数据结构。

在 RAG 里，最常见的是 VectorStoreIndex，即基于向量相似度检索的索引。

8. Retriever

Retriever 是“检索器”。

它决定：

- 从哪个 index 查
 - 查多少条
 - 用什么策略查
 - 是否使用 metadata filter
 - 是否混合关键词和向量检索
-

9. Node Postprocessor

Node Postprocessor 是“检索结果加工器”。

它对检索出来的 Node 做过滤、重排、扩展上下文等操作。

10. Response Synthesizer

Response Synthesizer 是“答案合成器”。

它把检索结果和用户问题交给 LLM，让 LLM 生成最终回答。

五、把 LlamaIndex RAG Pipeline 串起来看

完整流程可以理解为：

原始数据

↓

Loading: 加载数据



六、不同模块之间的关系

可以用一个表来总结：

阶段	主要对象	作用	通俗理解
Loading	Reader / Connector / Document	把外部数据加载进系统	搬资料
Transformation	Splitter / Metadata / Embedding	清洗、切块、加标签、向量化	整理资料
Indexing	VectorStoreIndex / SummaryIndex	建立可查询结构	建目录
Storing	StorageContext / Vector Store	保存索引和向量	存档案
Querying	QueryEngine / Retriever	响应用户问题	查资料
Postprocessing	Node Postprocessor	过滤、重排检索结果	精筛资料
Response Synthesis	Response Synthesizer / LLM	生成最终答案	写回答
Evaluation	Metrics / Test Set	评估效果	做质检

七、实际搭建 RAG 时的工程重点

1. 数据质量比模型本身更重要

RAG 的效果很大程度取决于数据处理质量。

如果 PDF 解析混乱、chunk 切得不好、metadata 缺失，那么后面的检索和回答都会受影响。

尤其是金融、法律、制度类场景，建议重点关注：

- 页码保留
- 表格解析
- 标题层级
- 文档来源
- 日期
- 权限
- 业务分类
- 原文引用

2. Chunk 策略会直接影响检索质量

常见问题：

- chunk 太大：检索结果不精准，噪声多
- chunk 太小：上下文断裂，模型回答不完整
- overlap 太少：跨段信息容易丢
- overlap 太多：重复信息增加成本

适合从一个中等配置开始，例如：

```
chunk_size = 512 或 800  
chunk_overlap = 50 或 100
```

然后通过 Evaluation 调整。

3. Metadata 是高质量 RAG 的关键

纯向量检索有时会召回语义相关但业务上不合适的内容。

例如用户问：

2024 年机构业务 KPI 规则是什么？

如果没有 metadata，系统可能检索到 2021 年、2022 年或其他业务线的制度。

如果有 metadata，就可以先过滤：

```
year = 2024
business = institutional
doc_type = KPI_policy
```

再做语义检索。

4. Top-K 不是越大越好

top_k 太小，可能漏掉关键信息。

top_k 太大，可能引入无关内容，导致回答混乱、成本上升。

一般可以从：

```
top_k = 3 到 10
```

开始测试。

文档中的查询示例就展示了可以把 similarity_top_k 设置为 10，并进一步用相似度阈值过滤。

5. Postprocessing 能显著提升质量

在实际系统中，单纯向量检索往往不够。

建议加入：

- similarity cutoff
- keyword filter
- metadata filter
- reranker
- prev-next context augmentation

这一步就像把“粗检索结果”再交给一个质检员筛一遍。

6. 存储是生产系统必需项

如果只是做 demo，可以每次临时建 index。

但如果是正式系统，必须持久化：

- 文档原文
- Node
- Metadata
- Embedding
- Vector index
- 文档版本信息

否则每次启动都重新建索引，不仅慢，而且贵。LlamaIndex 文档也强调，默认 indexed data 只存在内存中，因此通常需要存储，避免重复 re-index。

八、一个最小可行 RAG 系统应该长什么样？

如果你要用 LlamaIndex 做一个最小可行 RAG 系统，核心结构可以是：

1. 用 `SimpleDirectoryReader` 读取本地资料
2. 用 `SentenceSplitter` 或 `TokenTextSplitter` 切分文本
3. 给 `Document` / `Node` 加 `metadata`
4. 用 `VectorStoreIndex` 建立向量索引
5. 用 `Chroma` 或其他 `Vector Store` 存储 `embedding`
6. 用 `index.as_query_engine()` 创建查询引擎
7. 查询时设置 `top_k`、`similarity_cutoff`
8. 输出答案和 `source_nodes`
9. 用测试问题集评估准确性和忠实性

这就是从 demo 走向可用系统的基本骨架。

九、从学习角度，应该怎么掌握这套内容？

真正掌握 LlamaIndex RAG pipeline，不是背出这些名词，而是能够回答下面这些问题：

1. 为什么 RAG 需要 Loading、Indexing、Storing、Querying、Evaluation 这些阶段？
2. `Document` 和 `Node` 有什么区别？
3. 为什么要 chunk？`chunk_size` 和 `chunk_overlap` 会影响什么？

4. Embedding 为什么能支持语义检索？
5. VectorStoreIndex 和 SummaryIndex 分别适合什么场景？
6. top_k 是什么？为什么不是越大越好？
7. Retriever、Postprocessor、Response Synthesizer 分别负责什么？
8. 为什么生产系统必须持久化 index 和 embedding？
9. 如何通过 metadata filter 提高检索准确性？
10. 如何评估一个 RAG 系统是否真的有效？

能够把这些问题讲清楚，并自己搭出一个最小系统，就说明已经理解了 RAG pipeline 的主干。

十、一句话总结

LlamaIndex 的 RAG Pipeline 本质上是一套“把外部资料变成可检索知识，再让大模型基于检索结果回答问题”的工程流程。

它的核心链条是：

加载数据 → 切分和向量化 → 建立索引 → 存储索引 → 检索相关内容 → 后处理 → 合成答案
→ 评估效果

真正做项目时，最重要的不是只会调用 `index.as_query_engine()`，而是理解每一步对最终回答质量的影响：

数据怎么进来、怎么切、怎么存、怎么查、怎么筛、怎么让模型基于证据回答。